

CSCE 713 SOFTWARE SECURITY · MODULE 08 · LECTURE NOTES

Fuzzing

The technique that actually finds memory defects.

MODULE OUTCOMES

- Explain coverage-guided fuzzing and why it works.
- Write an effective harness.
- Measure a campaign properly.
- Explain structure-aware fuzzing and when it is needed.
- Explain what fuzzing cannot find.

1 Why coverage guidance works

COVERAGE FEEDBACK TURNS RANDOM MUTATION INTO A SEARCH

Random inputs almost never reach deep code — a parser rejects them at the first byte, so undirected random fuzzing explores the error-handling path exhaustively and the rest of the program not at all. **This is why fuzzing was considered a low-yield technique for years.**

Coverage guidance fixes it: instrument the program to report which edges executed, keep any input that reached a new edge, and mutate the kept inputs further — so progress accumulates rather than being rediscovered each time.

Which converts a random search into a hill-climb on coverage — and the consequence is striking: a magic number or a format signature is discovered byte by byte, because each correct byte opens a new edge and the input is therefore kept. The fuzzer reconstructs the format without being told it.

And it requires no specification whatsoever. The oracle is simply 'did it crash', which is why this works on code nobody documented and nobody understands — and why sanitisers multiply its power (Module 09), since they convert silent corruption into a crash.

- **A crash or a hang** — the default oracle, and the weakest one, **because a great many memory defects do not crash**: an out-of-bounds read returns whatever was there and execution continues.
- **A sanitiser report** — **which is what actually makes fuzzing effective. AddressSanitizer turns a silent one-byte out-of-bounds read into a loud, precisely located failure** (Module 09 §1).
- **An assertion** — your own invariants, checked on every single input, **which is the cheapest available way to broaden the oracle** and finds logic defects a crash oracle never would.

- **A differential comparison** — two implementations of the same format or protocol disagreeing on an input, **which finds correctness defects with no crash at all** and is how a great many parser and cryptographic implementation bugs were found.
- **And a round-trip property** — parse, then serialise, then parse again, which must be stable. **Fuzzing without sanitisers finds a fraction of what it should**, and **the pairing is not optional** — it is the most common omission in campaigns that report disappointing results.

2 The harness

```
int LLVMFuzzerTestOneInput(const uint8_t *d, size_t n) {
    Parser p;
    p.parse(d, n);    // the real entry point
    return 0;
}
```

RULES

```
target the REAL parsing entry point, not a
  validation wrapper or a convenience API
be deterministic -- no time, no randomness, no
  network, no filesystem; seed any PRNG from
  the input itself
be fast -- thousands of executions per second; a
  slow harness is a small campaign, and the
  relationship is linear
free everything, so that leak detection is
  meaningful
and do NOT catch the exceptions you want to find
```

SEED CORPUS

```
start from real, valid inputs -- a handful of
genuine files beats a million random bytes,
because coverage guidance needs somewhere to
climb from
```

Harness quality dominates campaign outcome, and it is worth saying explicitly because the instinct is to attribute a poor campaign to insufficient time. **The seed corpus is the most underrated input** — **real files give the fuzzer the format's grammar for free**, and a corpus of twenty genuine documents routinely outperforms a week of fuzzing from an empty corpus. **And targeting a validation wrapper rather than the real parser is the commonest harness defect**: the fuzzer spends its entire budget on the function that rejects malformed input, which is the one function you were not worried about.

3 Measuring

REPORT COVERAGE, NOT DURATION

'We fuzzed for 48 hours' says essentially nothing — a harness that rejects every input at byte zero can run forever and will have tested exactly one function. Duration is an input to the process, not a result of it.

So report the coverage achieved: which functions and which edges were reached, as a fraction of the target component — which is both measurable and, unlike duration, actionable.

And read the coverage specifically to find the blocked paths. An unreached branch usually means a checksum, a magic value, a length field, a decompression step, or a cryptographic check that the fuzzer cannot synthesise — and each of those has a specific fix (§3's list).

Which makes coverage the campaign's steering signal rather than a score — a plateau means change the harness or the corpus, not run it longer, and recognising that distinction is what separates a productive campaign from an expensive one. This is also what Project 2 grades, and why it asks for the coverage figure.

- Add seeds covering the formats, versions, and features you have not reached — the cheapest intervention available, and usually the most effective one.
- Disable checksum verification in the fuzz build, so that the fuzzer is not required to compute a valid checksum before reaching any parsing logic — and keep it enabled in production, since this is a build configuration rather than a code change.
- Split one harness into several, each targeting a different entry point or a different format, so that the search is not diluted across unrelated code.
- Use a dictionary of the format's tokens and keywords, so the mutator inserts meaningful strings directly rather than discovering each one byte by byte — which is nearly free and helps substantially on text formats.
- And go structure-aware if the format is deeply structured (§4). Disabling checksums in the fuzz build is the highest-yield single change for binary formats, and it is entirely safe because it cannot affect the shipped artefact.

4 Limits, and continuous operation

FUZZING FINDS CRASHES, SO IT CANNOT FIND ANYTHING THAT DOES NOT CRASH

Not found at all: missing authorisation checks, business logic errors, weak cryptography, information disclosure without a crash, and anything at all where the wrong answer is still a well-formed answer — which includes most of CSCE 701's subject matter.

Found poorly: defects requiring a specific sequence of operations rather than a single input, and concurrency defects, because the interleaving is not part of the input and therefore not part of what the fuzzer varies (Module 05 §4).

And deep state is genuinely hard — a defect reachable only after authentication, a session setup, and three prior requests in a particular order is not a single-input target, and no amount of mutation on one buffer reaches it.

So extend the oracle where you can: assertions over your own invariants, differential testing against a second implementation, and stateful harnesses that interpret the input buffer as a *sequence of operations* rather than as one document — which broadens the technique considerably and is how protocol implementations are fuzzed effectively.

- **Fuzz in CI on every change, briefly** — a few minutes per harness, which catches regressions in code that was just written and is cheap enough nobody objects.
- **And run long campaigns continuously and separately**, because **the interesting findings arrive after hours or days rather than minutes** — the two uses have different purposes and different infrastructure.
- **Keep the corpus** — it is the accumulated result of every previous campaign, and discarding it restarts the search from nothing. This is **the practice most often skipped**, and keeping it is what makes the next campaign start where the last one stopped.
- **Add every crash as a minimised regression test**, so that the specific defect cannot return silently — and minimisation is automated by every modern fuzzer.
- **And for open-source projects, OSS-Fuzz runs this entire infrastructure free** — continuous fuzzing, corpus management, crash triage, and regression tracking — **which removes the main practical obstacle to doing this properly** and has found tens of thousands of real defects by exactly the method described here.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
The Fuzzing Book (free, interactive)	The whole module, executable — build a coverage-guided fuzzer yourself, which is the fastest route to understanding §1.
libFuzzer and AFL++ documentation (free)	§2 and §3 in practice — the harness rules and the coverage tooling, from the implementations.

Resource	What it covers for this module
OSS-Fuzz, and its ideal integration guide (free)	§4's continuous operation , plus the best available guidance on harness design at scale.
Klees et al. — Evaluating Fuzz Testing (free)	§3's argument, rigorously — why duration and crash counts are poor measures, and what to report instead.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write a coverage-guided fuzzer** for a toy parser, following The Fuzzing Book.
- 2 **Show it discovering a four-byte magic number**, and explain why it can.
- 3 **Fuzz a parser without sanitisers, then with them**, and compare the findings.
- 4 **Write a harness that targets a validation wrapper**, then retarget it at the real parser and compare coverage.
- 5 **Fuzz from an empty corpus and from twenty real files**, and compare.
- 6 **Report coverage rather than duration** for one campaign.
- 7 **Find one unreachable branch** and determine what blocks it.
- 8 **Disable a checksum in the fuzz build** and measure the coverage change.
- 9 **Write a stateful harness** interpreting the input as an operation sequence.
- 10 **List three defect classes your campaign could not have found.**

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why do random inputs fail to reach deep code?
- 2 How does coverage guidance change that, and how is a magic number found?
- 3 Name five oracles and say which makes fuzzing effective.
- 4 Give five harness rules and the commonest harness defect.
- 5 Why does the seed corpus matter so much?
- 6 Why is duration a poor measure, and what should you report?
- 7 How do you read coverage to improve a campaign?
- 8 Give five responses to a coverage plateau.

- 9 What can fuzzing not find, and what does it find poorly?
- 10 How do you extend the oracle, and why keep the corpus?